Estructuras de Datos

2026
Primer Cuatrimestre

REPASO

Queriendo hacer el tren más grande

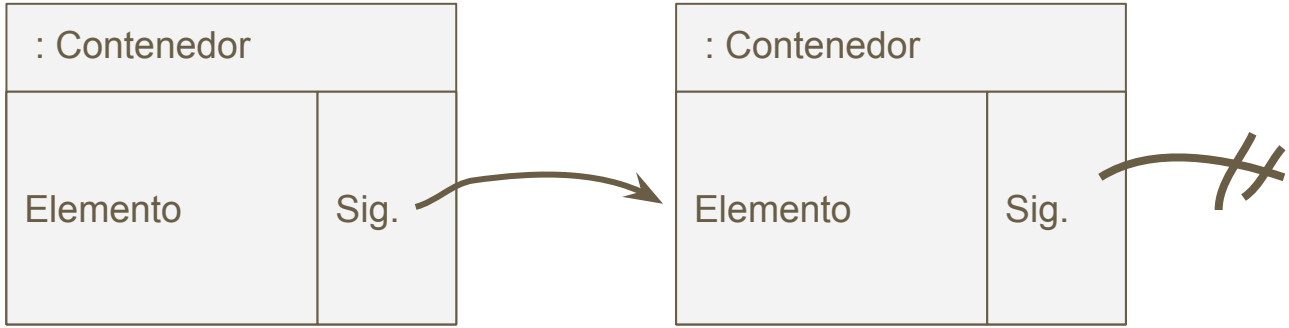
Insertar un vagón dentro del tren





Estructuras Enlazadas

Imaginemos un contenedor, una caja donde podemos guardar un elemento. Y esa misma caja se enlaza (se conecta como los vagones de un tren) con otra caja y así sucesivamente



Node<E>

```
public class Node<E> {
    protected E element;
    protected Node<E> next;
    public Node( E e ) {...}
    public E getElement() {...}
    public void setElement( E element ) {...}
    public Node<E> getNext() {...}
    public void setNext( Node<E> e ) {...}
}
```

TDA Pila con Nodo<E>

Node.java | MiPila.java | TesterMiPila.java | MiPila.java

```
1  package ed.pila;
2  import java.util.LinkedList;
5
6  public class MiPilaNodo<E> extends Stack<E> {
7
8      protected Node<E> tope;
9      protected int cantidad;
10
11     /**
12      * Crea una pila vacía
13      */
14     public MiPilaNodo() {
15         this.tope = null;
16         this.cantidad = 0;
17     }
18
19     /**
20      * Retorna verdadero si la pila está vacía, falso en ca
21      */
22     public boolean empty() {
23         return (this.cantidad == 0);
24     }
25
26     /**
27      * Retorna el elemento en la parte superior de esta pil
28      */
29     public E top () {
30         E resultado = null;
31         if(!this.empty()) {
32             resultado = this.tope.getElement();
33         }
34         return resultado;
35     }
36
```

TDA Cola con Nodo<E>

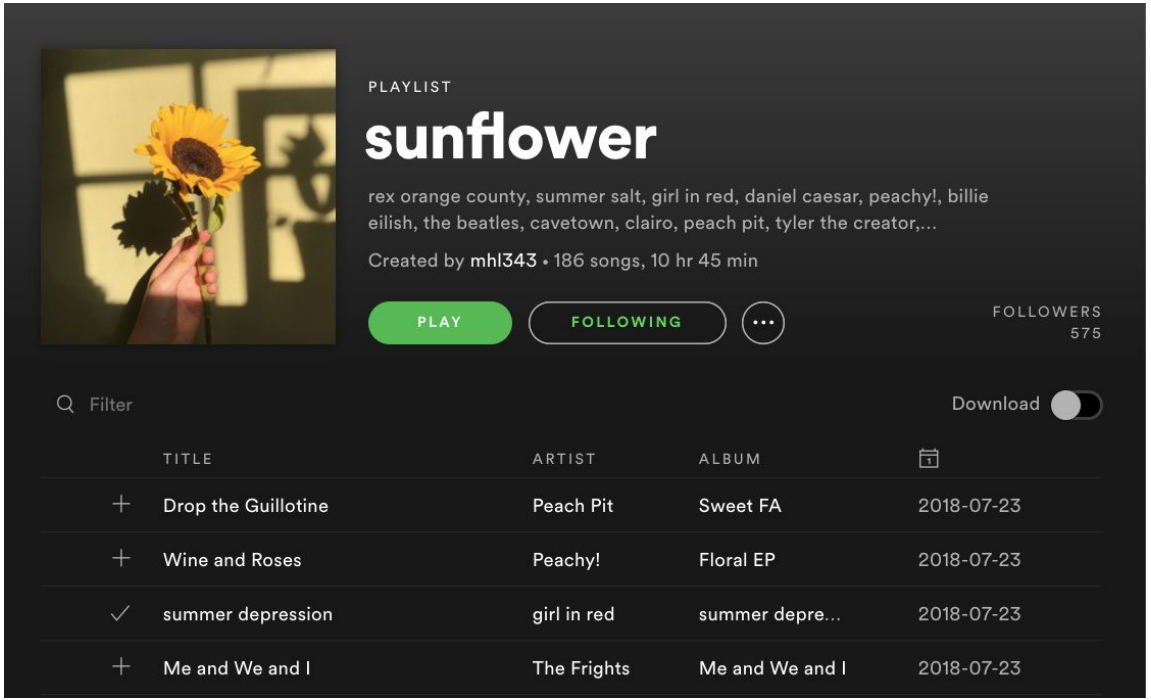
```
public class MiQueueNode<E> implements Queue<E> {
    protected Node<E> frente; // por acá saco
    protected Node<E> fondo; // por acá inserto
    protected int cantidad;

    public MiQueueNode() {
        this.frente = null;
        this.fondo = null;
        this.cantidad = 0;
    }

    @Override
    public boolean isEmpty() {
        return this.cantidad == 0;
    }
```

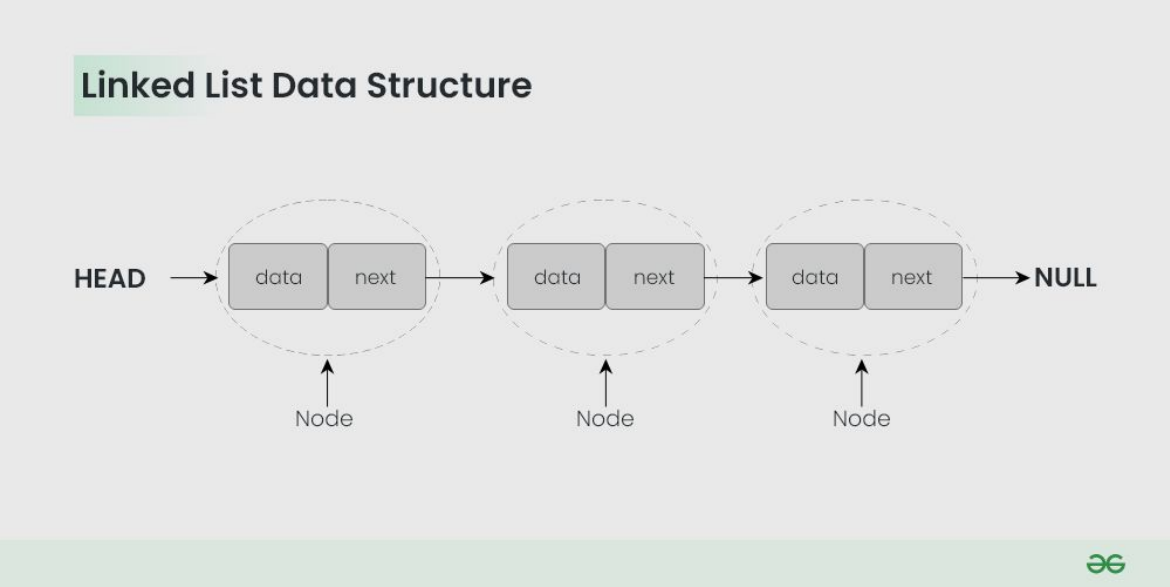
TDA Lista

TDA Lista



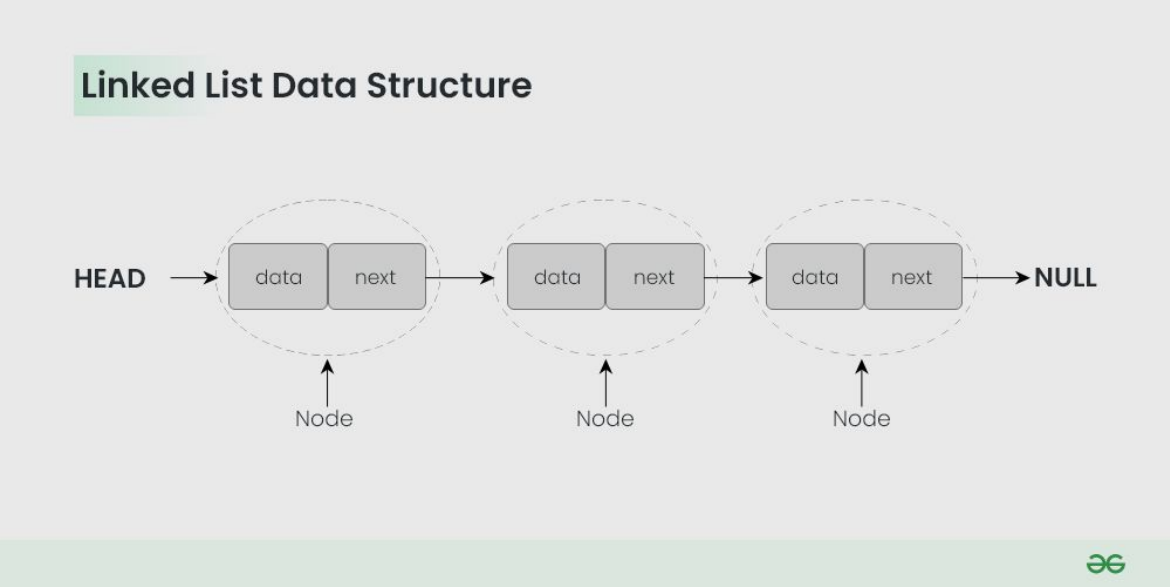
En una playlist yo puedo reproducir cualquier tema de la lista. Además, puedo modificar mi playlist en cualquier ubicación de la misma.

TDA Lista



En una playlist yo puedo reproducir cualquier tema de la lista. Además, puedo modificar mi playlist en cualquier ubicación de la misma.

TDA Lista



En una **lista** yo puedo **acceder** a cualquier elemento. Además, puedo **modificar** mi lista en cualquier **posición** de la misma.

TDA Lista

Una lista es una estructura de datos lineal que permite ser modificada en cualquier ubicación. Esta libertad requiere que tengamos una forma de señalar dentro de la lista, lo que vamos a llamar **posición**.

TDA Lista

Es natural pensar a la **posición** como un índice; a partir de lo trabajado en IPOO pueden pensar a la posición como un índice dentro de un arreglo. O sea, posición podría ser un entero. Pero...

TDA Lista

Es natural pensar a la **posición** como un índice; a partir de lo trabajado en IPOO pueden pensar a la posición como un índice dentro de un arreglo. O sea, posición podría ser un entero. Pero...

¿Abstracción?

¿Encapsulamiento?

¿Modularidad?

TDA Lista

Vamos a tratar a la **posición** como un elemento abstracto. Es decir, vamos a tener un tipo para **posición** y otro tipo para **lista**

TDA Lista con Posición (PositionList)

Estas son operaciones de PositionList que involucran una Position

first(): retorna la posición del primer elemento de la lista; error si la lista está vacía.

last(): retorna la posición del último elemento de la lista; error si la lista está vacía

prev(p): retorna la posición del elemento que precede al elemento en la posición p; error si p = first()

next(p): retorna la posición del elemento que sigue al elemento en la posición p; error si p = last()

TDA Lista con Posición (PositionList)

```
position p = lista.first();
position fin = lista.last();
while(p != fin) {
        // hago algo con el elemento en p
        p = lista.next(p);
}
// hago algo con el último elemento
```

TDA Lista con Posición (PositionList)

Cualquier cosa que sea una **position**, nos tiene que dar la posibilidad de acceder al elemento ubicado en ese lugar

```
public interface Position<E> {
    // Retorna el valor del elemento ubicado en la posición
    public E element();
}
```

TDA Lista con Posición (PositionList)

```
Position<E> p = lista.first();
Position<E> fin = lista.last();
while(p != fin) {
        E element = p.element();
        // hago algo con el elemento en p
        p = lista.next(p);
}
E element = p.element();
// hago algo con el último elemento
```

TDA Lista con Posición (PositionList)

first(): retorna la posición del primer elemento de la lista; error si la lista está vacía.

last(): retorna la posición del último elemento de la lista; error si la lista está vacía

prev(p): retorna la posición del elemento que precede al elemento en la posición p; error si p = first()

next(p): retorna la posición del elemento que sigue al elemento en la posición p; error si p = last()

TDA Lista con Posición (PositionList)

first(): retorna la posición del primer elemento de la lista; error si la lista está vacía.

last(): retorna la posición del último elemento de la lista; error si la lista está vacía

prev(p): retorna la posición del elemento que precede al elemento en la posición p; error si p = first()

next(p): retorna la posición del elemento que sigue al elemento en la posición p; error si p = last()

... ahora modifiquemos la lista...

TDA Lista con Posición (PositionList)

Estas son operaciones de PositionList que involucran Position y el elemento paramétrico

set(p, e): Reemplaza al elemento en la posición p con e, retornando el elemento que estaba antes en la posición p

addFirst(e): Inserta un nuevo elemento e como primer elemento

addLast(e): Inserta un nuevo elemento e como último elemento

addBefore(p, e): Inserta un nuevo elemento e antes de la posición p

addAfter(p, e): Inserta un nuevo elemento e luego de la posición p

remove(p): Elimina y retorna el elemento en la posición p invalidando la posición p

TDA Lista con Posición

Implementemos un método que reciba una PositionList genérica de tipo E, y dos elementos e1 y e2. El método deberá reemplazar todas las apariciones del elemento e1 por e2.

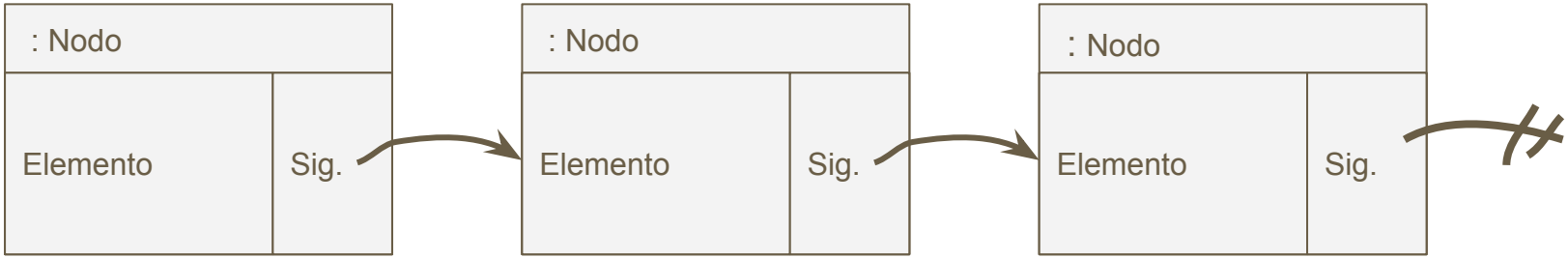
TDA Lista con Posición

```
método foo(original, nuevo, lista) { // lista nunca es vacía
position p = lista.first();
while( p != lista.last() ) {
        if(p.element() == original) { lista.set(p, nuevo); }
        p = lista.next(p);
}
if(p.element() == original) { lista.set(p, nuevo); }
```

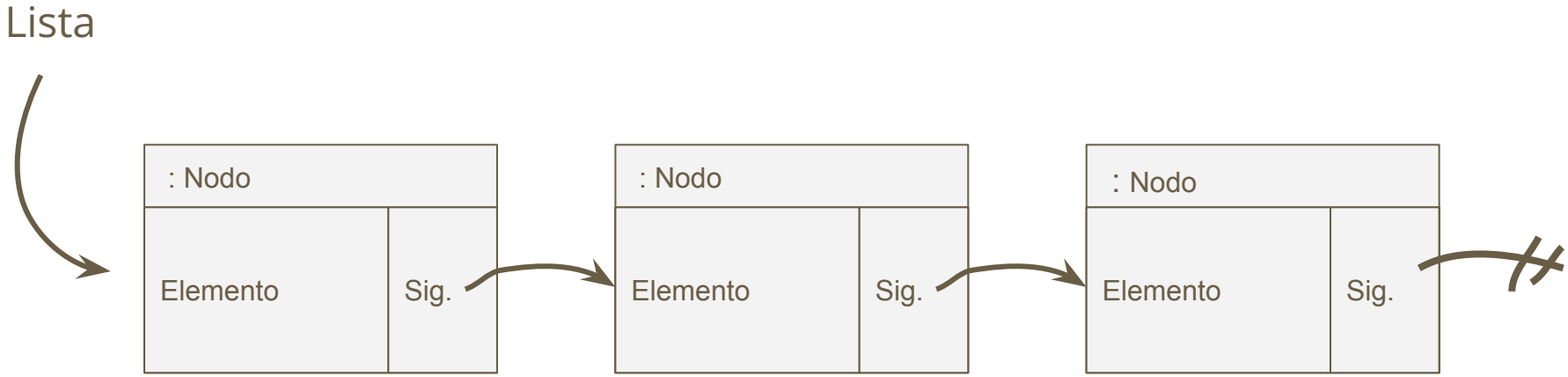
TDA Lista con Nodo<E>

+ Position

TDA Lista con Nodo<E> + Position

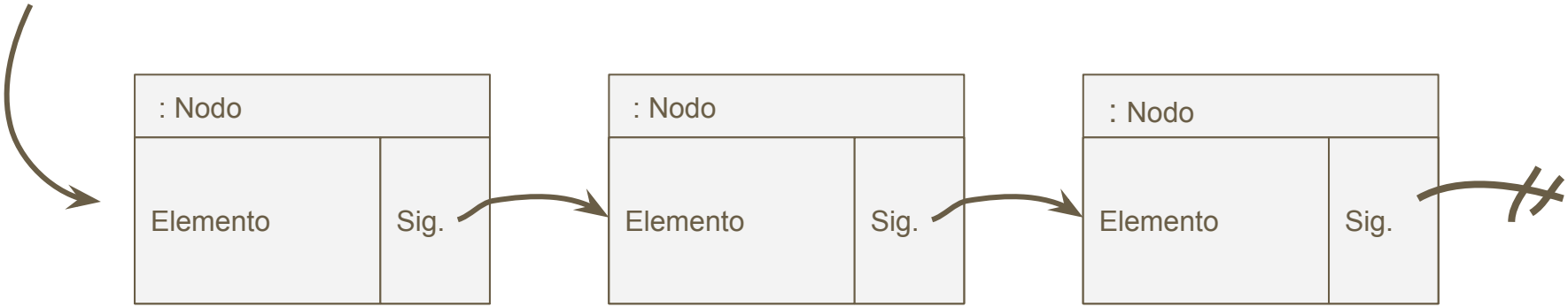


TDA Lista con Nodo<E> + Position



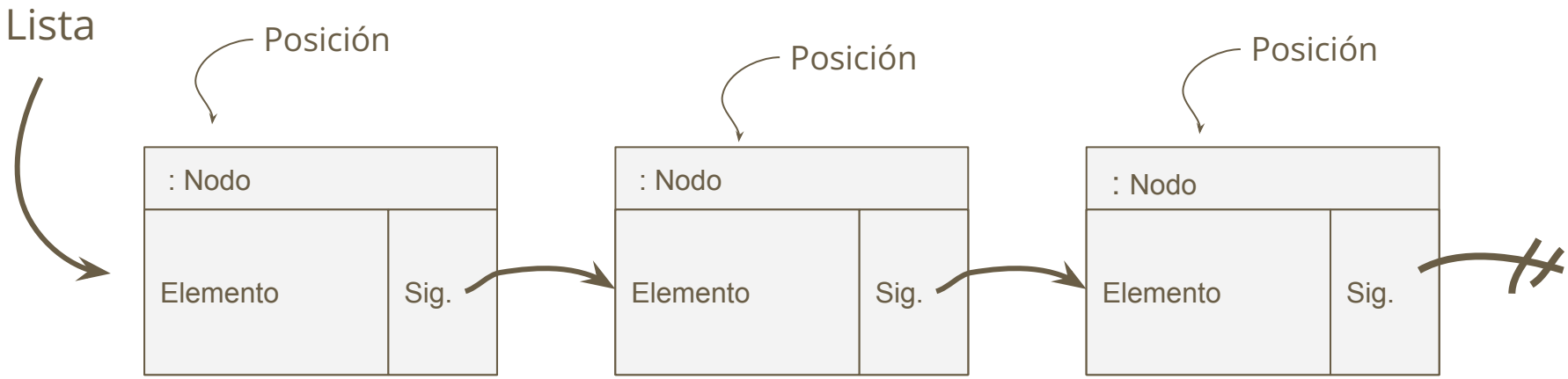
TDA Lista con Nodo<E> + Position

Lista



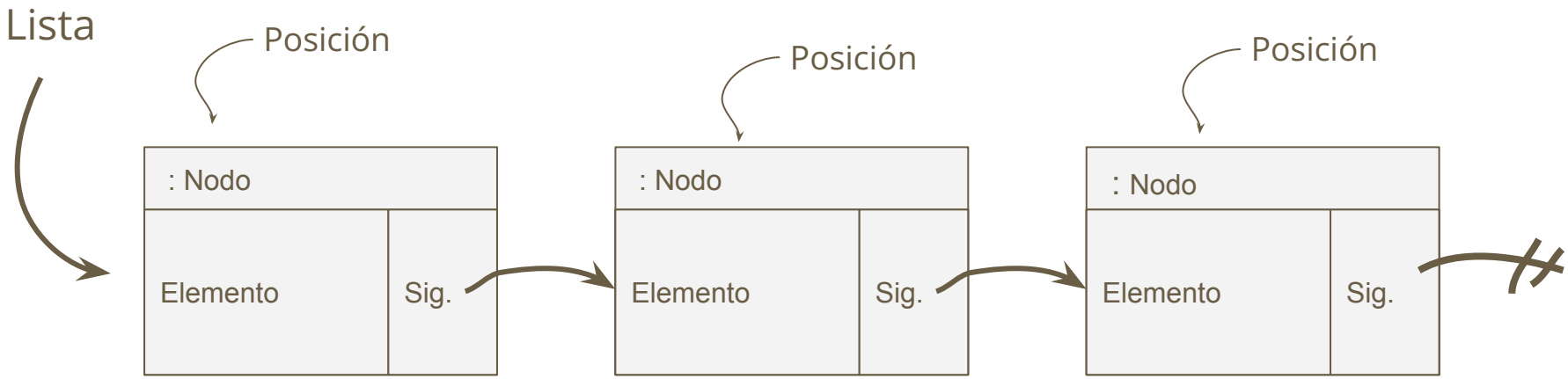
Una Lista es una secuencia de nodos enlazados

TDA Lista con Nodo<E> + Position



Una Lista es una secuencia de nodos enlazados

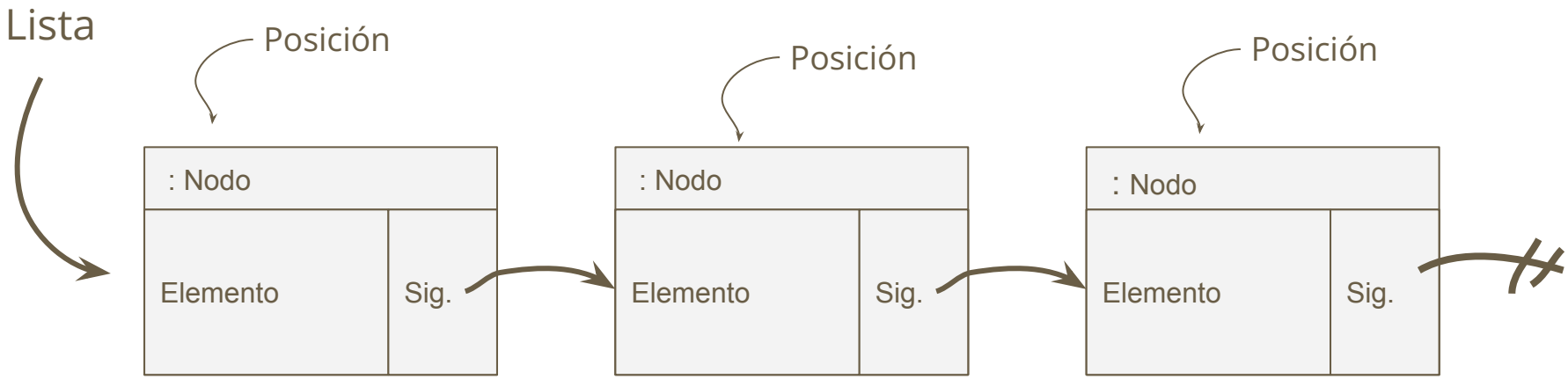
TDA Lista con Nodo<E> + Position



Una Lista es una secuencia de nodos enlazados

Cada nodo juega el rol de una posición

TDA Lista con Nodo<E> + Position



Una Lista es una secuencia de nodos enlazados

El Nodo **es** una Posición

TDA Lista con Nodo<E> + Position

```
public interface Position<E>{

        public E element();

}
```

TDA Lista con Nodo<E> + Position

```java
public class Node<E> implements Position<E>{
    protected E elem;
    protected Node<E> next;

    public Node( E e ) {
        this.elem = e;
        this.next = null;
    }

    @Override
    public E element() {
        return elem;
    }

    public void setElement(E elem) {
        this.elem = elem;
    }

    public Node<E> getNext() {
        return next;
    }

    public void setNext(Node<E> next) {
        this.next = next;
    }
}
```

TDA Lista con Nodo<E> + Position

```
8
9  public class ListaEnlazada<E> implements PositionList<E> {
10
11     //atributos
12     protected int cantElems;
13     protected Node<E> header;
14
15     @Override
16     public int size() {
17         return this.cantElems;
18     }
19
20     @Override
21     public boolean isEmpty() {
22         return this.cantElems == 0;
23     }
24
25     @Override
```

TDA Lista con Nodo<E> + Position

Situaciones de error relacionadas a una posición p

p = null

p fue eliminada previamente de la lista

p es una posición de una lista diferente

p es la primera posición de la lista e invocamos a prev(p)

p es la última posición de la lista e invocamos a next(p)

TDA Lista con Nodo<E> + Position

Situaciones de error relacionadas a una posición p

p = null

p fue eliminada previamente de la lista

p es una posición de una lista diferente

p es la primera posición de la lista e invocamos a prev(p)

p es la última posición de la lista e invocamos a next(p)

es posible validar (2) seteando el rótulo del nodo en null al eliminar la posición

podría ser validado recorriendo la lista para ver si la posición es válida o haciendo que el nodo conozca la lista que lo contiene. Por cuestiones de eficiencia no lo haremos

TDA Lista con Nodo<E> + Position

```
private Node<E> checkPosition(Position<E> p) throws InvalidPositionException {
Node<E> n;
try {
    n= (Node<E>) p;
} catch( ClassCastException e ) {
        throw new InvalidPositionException("Posicion invalida");  }
return n;
}
```

TDA Lista con Nodo<E> + Position

Esta implementación que acabamos de presentar se conoce como lista **simplemente enlazada**. Observen que los nodos solo “conocen” al siguiente.

¿Qué problemas trae esta implementación?

BREAK

TDA Lista con Nodo<E> + Position

Esta implementación que acabamos de presentar se conoce como lista **simplemente enlazada**. Observen que los nodos solo “conocen” al siguiente.

¿Qué problemas trae esta implementación?

Lista con Nodo<E> Simplemente Enlazada + Posición		
Métodos del TDA	¿Se resuelve inmediato?	¿Salimos a buscar algo?
first()		
last()		
prev(p)		
next(p)		
set(p, e)		
addFirst(e)		
addLast(e)		
addBefore(p, e)		
addAfter(p, e)		
remove(p)		

Lista con Nodo<E> Simplemente Enlazada + Posición		
Métodos del TDA	¿Se resuelve inmediato?	¿Salimos a buscar algo?
first()	Si	
last()	No	
prev(p)	No	
next(p)	Si	
set(p, e)	Si	
addFirst(e)	Si	
addLast(e)	No	
addBefore(p, e)	No	
addAfter(p, e)	Si	
remove(p)	No	

Lista con Nodo<E> Simplemente Enlazada + Posición		
Métodos del TDA	¿Se resuelve inmediato?	¿Salimos a buscar algo?
first()	Si	No
last()	No	Si
prev(p)	No	Si
next(p)	Si	No
set(p, e)	Si	No
addFirst(e)	Si	No
addLast(e)	No	Si
addBefore(p, e)	No	Si
addAfter(p, e)	Si	No
remove(p)	No	Si

TDA Lista Mejorada

¿Qué problemática tienen en común todos estos métodos?

prev(p)

addBefore(p,e)

remove(p)

TDA Lista Mejorada

¿Qué problemática tienen en común todos estos métodos?

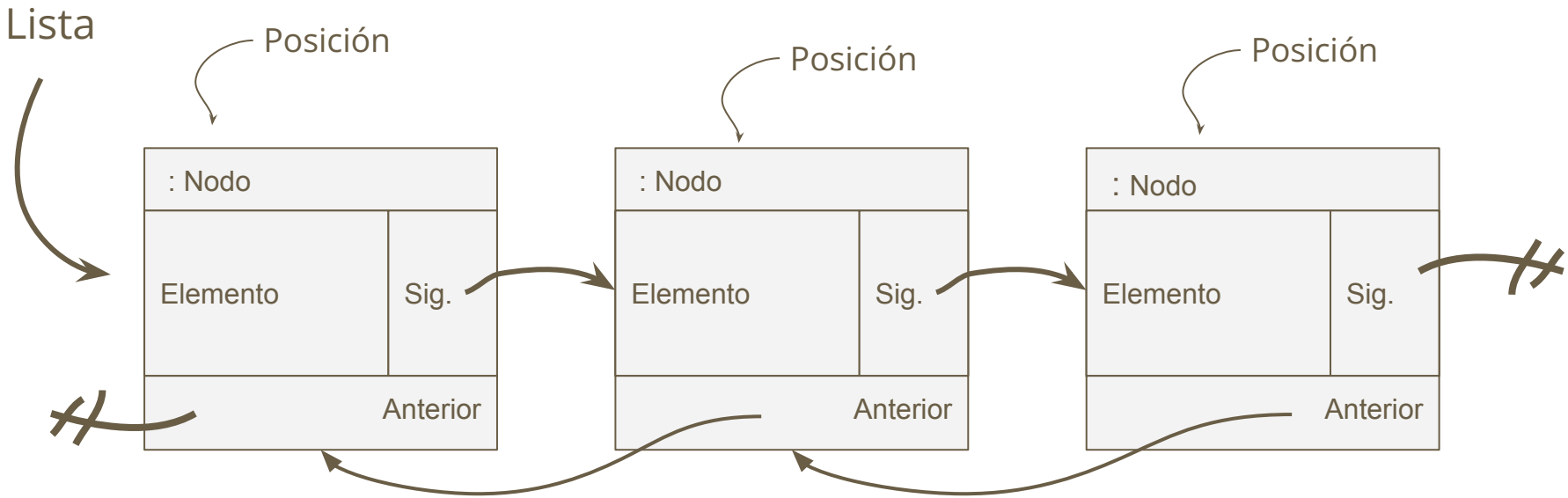
prev(p)

addBefore(p,e)

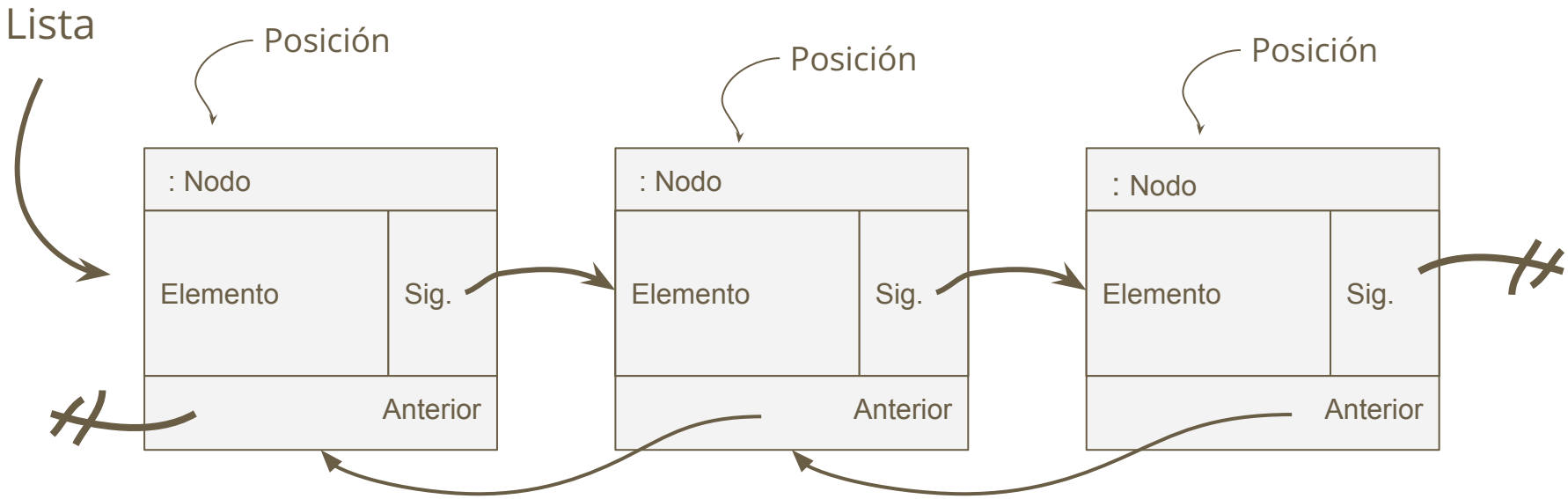
remove(p)

Necesitamos acceder a la posición anterior a la dada

TDA Lista con Nodo<E> + Position + Doblemente Enlazada



TDA Lista con Nodo<E> + Position + Doblemente Enlazada



Nodo Doblemente Enlazado

TDA Lista con Nodo<E> Doblemente Enlazado

+ Position

TDA Lista con Nodo<E> Doblemente Enlazado + Position

Noten que este cambio que hicimos en la estructura **NO** es un nuevo tipo de dato abstracto. Seguimos hablando del TDA Lista con Posición sólo que ahora tenemos una implementación más eficiente.

TDA Lista con Nodo<E> Doblemente Enlazado + Position

Noten que este cambio que hicimos en la estructura **NO** es un nuevo tipo de dato abstracto. Seguimos hablando del TDA Lista con Posición sólo que ahora tenemos una implementación más eficiente.

Y podemos seguir mejorándola

Lista con Nodo<E> Simplemente Enlazada + Posición		
Métodos del TDA	¿Se resuelve inmediato?	¿Salimos a buscar algo?
first()	Si	No
last()	No	Si
prev(p)	No	Si
next(p)	Si	No
set(p, e)	Si	No
addFirst(e)	Si	No
addLast(e)	No	Si
addBefore(p, e)	No	Si
addAfter(p, e)	Si	No
remove(p)	No	Si

Lista con Nodo<E> Simplemente Enlazada + Posición		
Métodos del TDA	¿Se resuelve inmediato?	¿Salimos a buscar algo?
first()	Si	No
last()	No	Si
prev(p)	Si	No
next(p)	Si	No
set(p, e)	Si	No
addFirst(e)	Si	No
addLast(e)	No	Si
addBefore(p, e)	Si	No
addAfter(p, e)	Si	No
remove(p)	Si	No

TDA Lista Mejorada

¿Qué problemática tienen en común todos estos métodos?

last()

addLast(e)

TDA Lista Mejorada

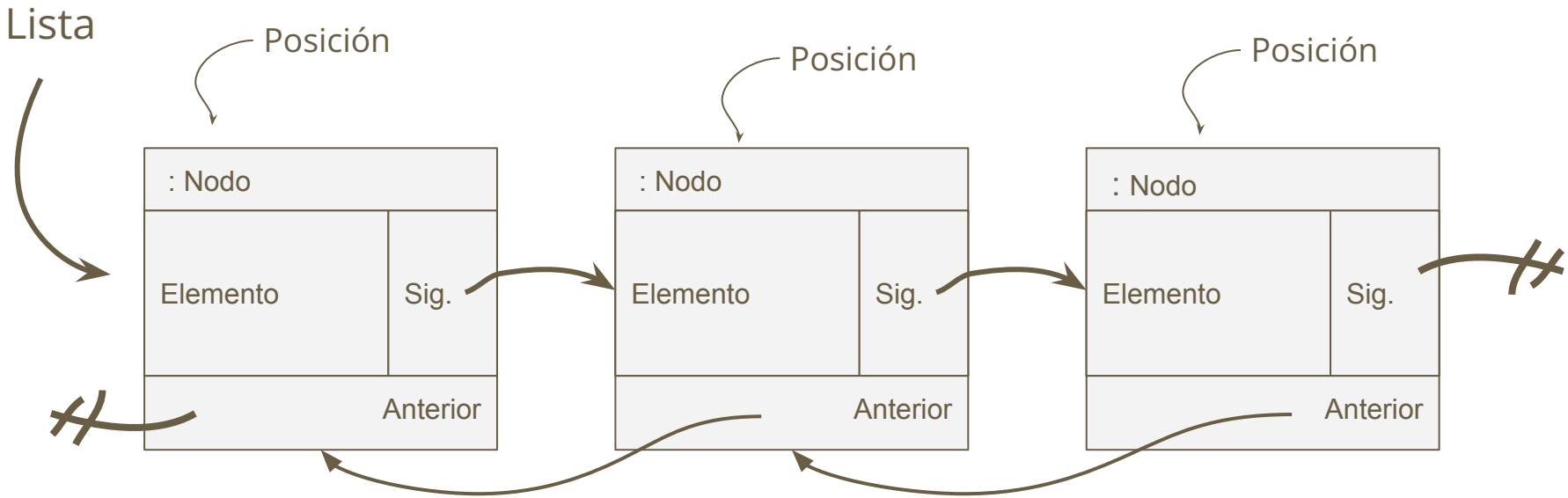
¿Qué problemática tienen en común todos estos métodos?

last()

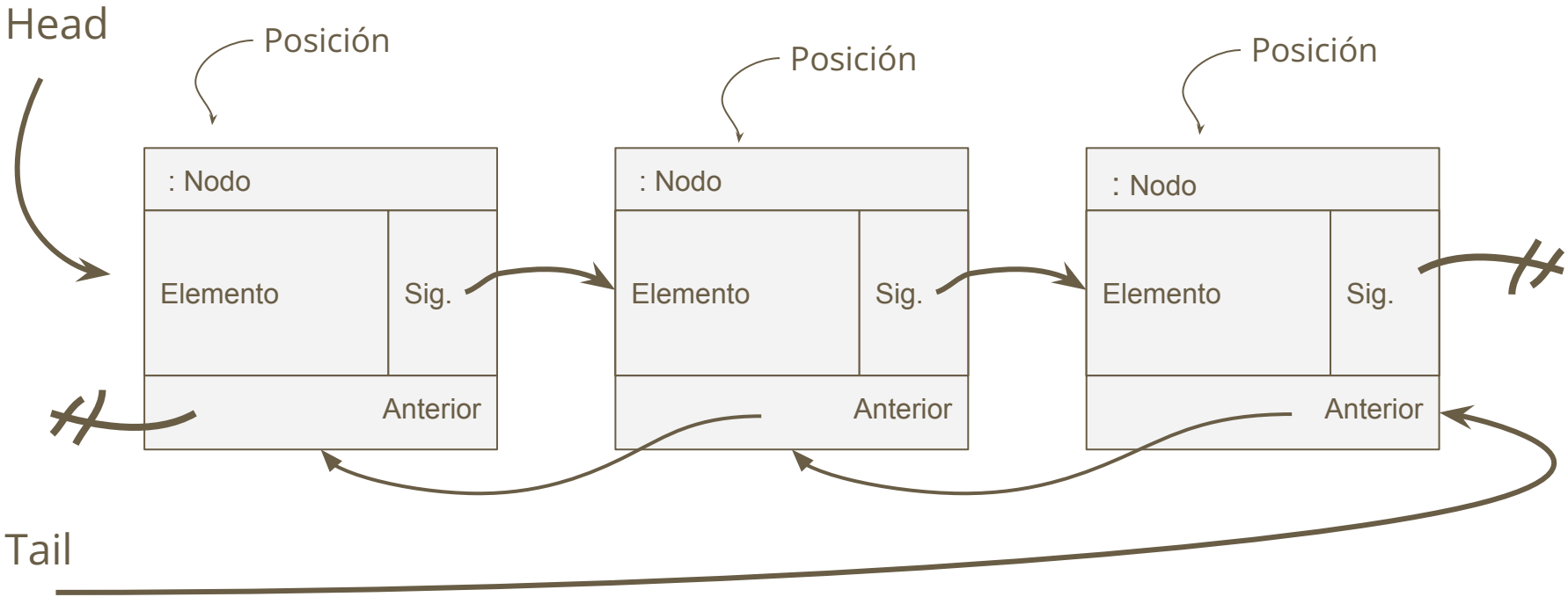
addLast(e)

Necesitamos acceder a la última posición

TDA Lista con Nodo<E> + Position + Doblemente Enlazada



TDA Lista con DNodo<E> + Position + Doblemente Enlazada con Enlace al Final



TDA Lista con DNodo<E> + Position + Doblemente Enlazada con Enlace al Final

¿Recuerdan cuando implementamos el TDA Cola con nodos enlazados, que una cola vacía generaba casos particulares al insertar (insertar en cola vacía) o eliminar (eliminar y dejar la cola vacía)?

TDA Lista con DNodo<E> + Position + Doblemente Enlazada con Enlace al Final con Centinelas

Para unificar ciertas operaciones vamos a implementar la lista doblemente enlazada con nodos centinelas. Esto significa que los atributos de instancia head y tail no van a referenciar a los nodos reales de la lista sino que a “centinelas”.

Estos nodos centinelas tienen la característica de tener el elemento nulo y se crean en el constructor. Por lo tanto cuando la lista está vacía solo tiene los centinelas head y tail enlazados :)

TDA Lista con DNodo<E> + Position + con Enlace al Final y con Centinelas

TDA Lista con DNodo<E> + Position + con Enlace al Final y con Centinelas

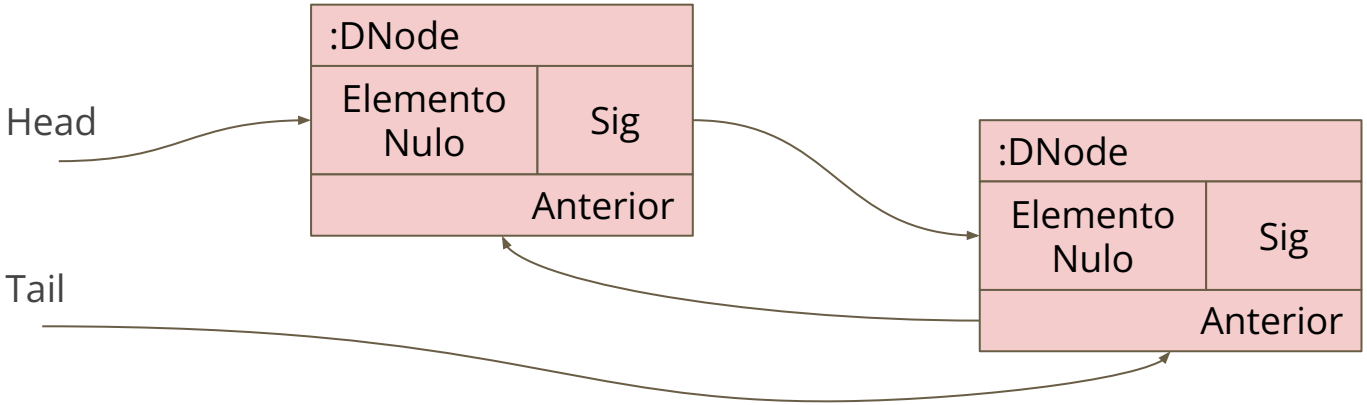
// Creo una lista vacía

Head

Tail

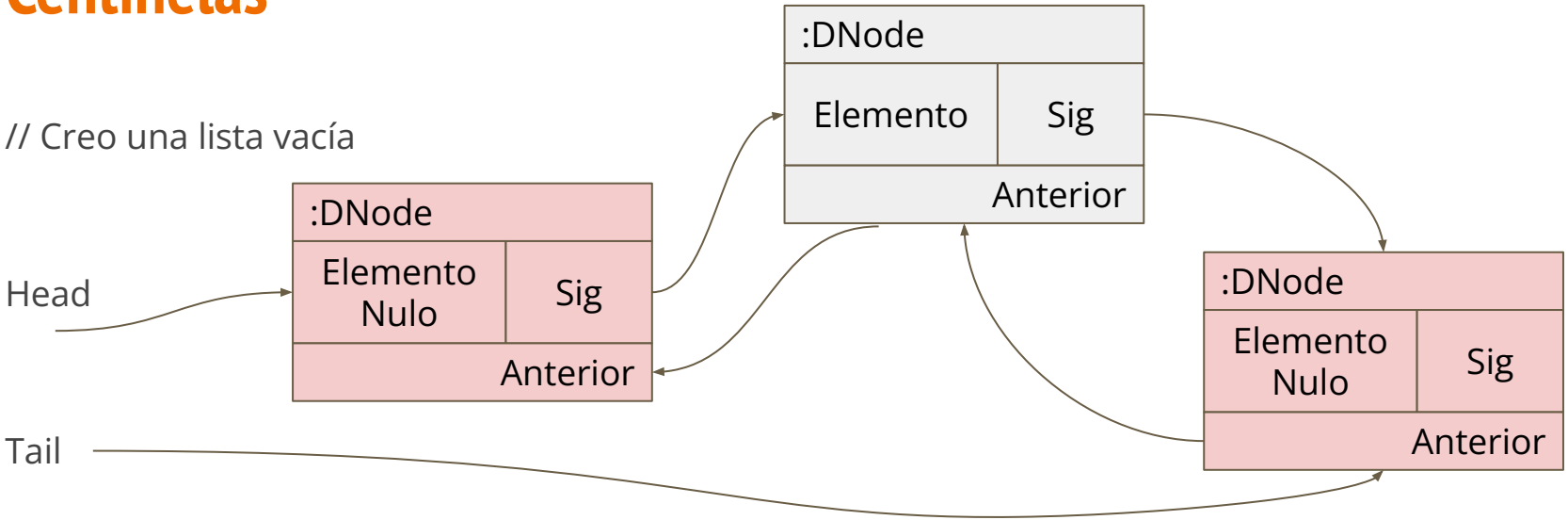
TDA Lista con DNodo<E> + Position + con Enlace al Final y con Centinelas

// Creo una lista vacía



Notar que la posición de los centinelas no son posiciones válidas para el cliente

TDA Lista con DNodo<E> + Position + con Enlace al Final y con Centinelas



Bibliografía

- Goodrich & Tamassia, capítulo 6